

CH 3.1: DISTRIBUTED QUERY OPTIMIZATION

Contents

Semi Join ($\bowtie$).....	1
Distributed Query Optimization	1
Direct Join Ordering.....	2
Semi Join Based Algorithm	3
System R Join Algorithm (Nested Loop Join with Optimizations).....	4
R* Algorithm.....	5
2 methods in R* for inter-site data transfer	6
Join Processing and Cost Formulas in R* Algorithm.....	6
Hill Climbing.....	9
SDD-1 Algorithm	9
Distributed Cost Model.....	9
Factors Governing Query Optimization	10
Centralized and Decentralized Query Optimization	11

Semi Join ($\bowtie$)

A semi join between two tables A and B returns only the rows from A that have a matching row in B, but does not return B's columns. It is useful in distributed environments to reduce data transfer between sites. Example:

Student (at Site A)

Sid	Sname	Tid
1	Ram	T1
2	Shyam	T2
3	Sita	T3
4	Radha	T1

Teacher (at Site B)

Tid	Tname
T1	Shiva
T2	Vishnu

Student $\bowtie$ Teacher (at site A)

Sid	Sname	Tid
1	Ram	T1
2	Shyam	T3
4	Radha	T1

Distributed Query Optimization

In DDBMS, join operations across different sites can cause significant communication costs, making join ordering crucial.

Why Join Ordering Matters

- **In centralized DBMS:** Join order affects performance (e.g., System R optimizer).
- **In distributed DBMS:** Data transfer cost is added, making efficient join planning even more critical.

Two Main Approaches

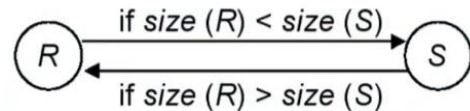
i. Direct Join Ordering

- Optimize the order in which joins are executed.
- Systems: **INGRES and Distributed INGRES, System R and System R***

- **Strategy:**
 - Move the smaller relation to the site of the larger one.
 - Estimate sizes of relations (R and S) to minimize data movement.
- ii. **Join Replacement using Semi joins**
 - Replace full joins with **semi join sequences** to reduce data transfer.
 - Systems: **Hill Climbing, SDD-1**
 - Especially effective when:
 - Relations are large
 - Selectivity is high (few matches)

Direct Join Ordering

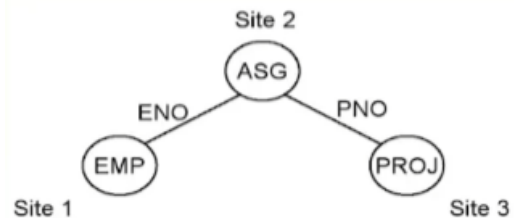
- If relation **R** is at **Site A**, and relation **S** is at **Site B**:
 - Move the **smaller** of R or S to the other site.
 - Perform the join locally after transfer.
- Direct join ordering of queries involving more than 2 relation is substantially more complex.



Example:

This query can be evaluated in the following ways:

- 1) EMP → Site 2
- 2) Site 2: EMP' = EMP ⋈ ASG
- 3) EMP' → Site 3
- 4) Site 3: EMP' ⋈ PROJ



- 1) PROJ → Site 2
- 2) Site 2: PROJ' = PROJ ⋈ ASG
- 3) PROJ' → Site 1
- 4) Site 1: PROJ' ⋈ EMP

- 1) ASG → Site 1
- 2) Site 1: EMP' = EMP ⋈ ASG
- 3) EMP' → Site 3
- 4) Site 3: EMP' ⋈ PROJ

- 1) ASG → Site 3
- 2) Site 3: PROJ' = PROJ ⋈ ASG
- 3) PROJ' → Site 1
- 4) Site 1: PROJ' ⋈ EMP

- 1) EMP → Site 2
- 2) PROJ → Site 2
- 3) Site 3: EMP ⋈ PROJ ⋈ ASG

To select a plan from above, a lot of information are needed:

- Size (EMP)
- Size (ASG)
- Size (PROJ)
- Size (EMP ⋈ ASG)
- Size (ASG ⋈ PROJ)

Use **direct join** only if:

- **R is already small**
- Simplicity is prioritized over communication cost

Semi Join Based Algorithm

Assuming: The join of 2 relation R and S over attributes R stored at sites 1 and 2 respectively

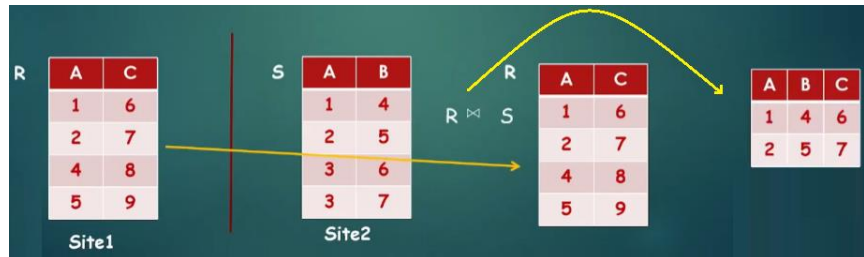
Cost Analysis:

- Direct Join ordering: $R \bowtie_A S$
- Semi join based algorithm: $(R \bowtie_A S) \bowtie S$

(assuming size (R) < size (S))

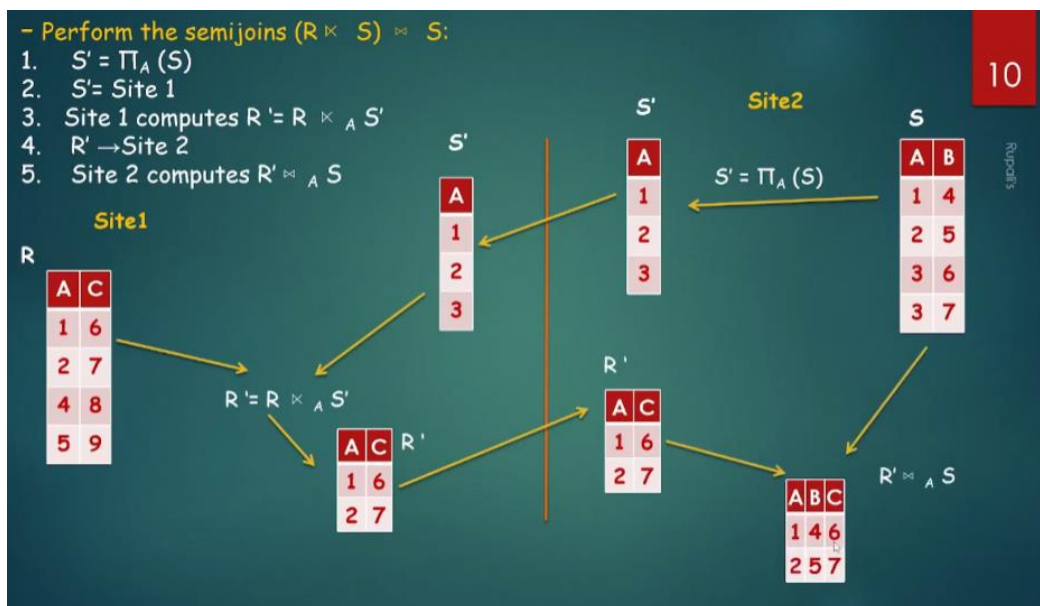
a) Perform the join $R \bowtie_A S$
(Direct join ordering)

- $R \rightarrow \text{site 2}$
- Site 2 computes $R \bowtie S$



b) Perform the semijoins $(R \bowtie S) \bowtie_A S$

- $S' = \pi_A(S)$
- $S' \rightarrow \text{Site 1}$
- Site 1 computes $R' = R \bowtie_A S'$
- $R' \rightarrow \text{Site 2}$
- Site 2 computes $R' \bowtie_A S$



- We can conclude that semi join is better if $\text{sizeof}(\pi_A(S)) + \text{size}(R \bowtie S) < \text{size}(R)$
- The semi join approach is better if it acts as a sufficient reducer (few tuples participate in join)
- **Semijoin-based strategy (b) is better** in most distributed environments, especially when:
 - **R is large**
 - **Join selectivity is low**
 - **We want to reduce network traffic**
- Use **direct join (a)** only if:
 - **R is already small**
 - **Simplicity is prioritized over communication cost**

System R Join Algorithm (Nested Loop Join with Optimizations)

The **System R** algorithm developed by IBM was a pioneering RDBMS that introduced a **cost-based optimizer** and practical join strategies. Its join algorithm is based on the **block nested loop join**, optimized using access paths (indexes, sort orders, etc.).

- **Cost-based:** Selects the join plan with the lowest estimated cost.
- **Dynamic programming based**
- **Semi-joins:** Used to reduce unnecessary data transfer.
- **Data localization:** Tries to perform operations at data source sites.
- **Communication-efficient:** Minimizes inter-site data movement.
- **Statistics-driven:** Relies on metadata like table size and selectivity.
- **Distributed join support:** Handles hash, sort-merge, and nested-loop joins.
- **Site autonomy:** Works with independently managed database sites.
- **Scalable:** Efficient with increasing sites and data size.
- **Multi-query optimization:** Shares results across multiple queries to avoid redundancy.
- **Adaptive execution:** Can adjust execution plan based on run-time info.

Join Strategy in System R

System R uses a dynamic programming approach and supports two main join algorithms:

1. Nested-Loop Join (NLJ)
2. Sort-Merge Join (SMJ)

It estimates the cost of each plan using statistics, and chooses the least-cost plan.

Assumptions:

- Relations **R** and **S** need to be joined on a condition (e.g., $R.A = S.B$)
- Multiple access paths (sequential scan, indexed access, etc.) may be available
- **Smaller relation is outer**

Example:

Names of employees working on CAD/CAM project

PROJ $\bowtie$ PNO ASG $\bowtie$ PNO EMP

EMP			ASG			
ENO	ENAME	TITLE	ENO	PNO	RESP	DUR
E1	J. Doe	Elect. Eng	E1	P1	Manager	12
E2	M. Smith	Syst. Anal.	E2	P1	Analyst	24
E3	A. Lee	Mech. Eng.	E2	P2	Analyst	6
E4	J. Miller	Programmer	E3	P3	Consultant	10
E5	B. Casey	Syst. Anal.	E3	P4	Engineer	48
E6	L. Chu	Elect. Eng.	E4	P2	Programmer	18
E7	R. Davis	Mech. Eng.	E5	P2	Manager	24
E8	J. Jones	Syst. Anal.	E6	P4	Manager	48
			E7	P3	Engineer	36
			E8	P3	Manager	40

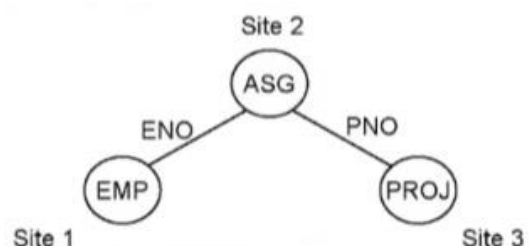
PROJ			PAY	
PNO	PNAME	BUDGET	TITLE	SAL
P1	Instrumentation	150000	Elect. Eng.	40000
P2	Database Develop.	135000	Syst. Anal.	34000
P3	CAD/CAM	250000	Mech. Eng.	27000
P4	Maintenance	310000	Programmer	24000

Answer

Indexes:

- EMP has an index on ENO
- ASG has an index on PNO
- PROJ has an index on PNO and an index on PNAME

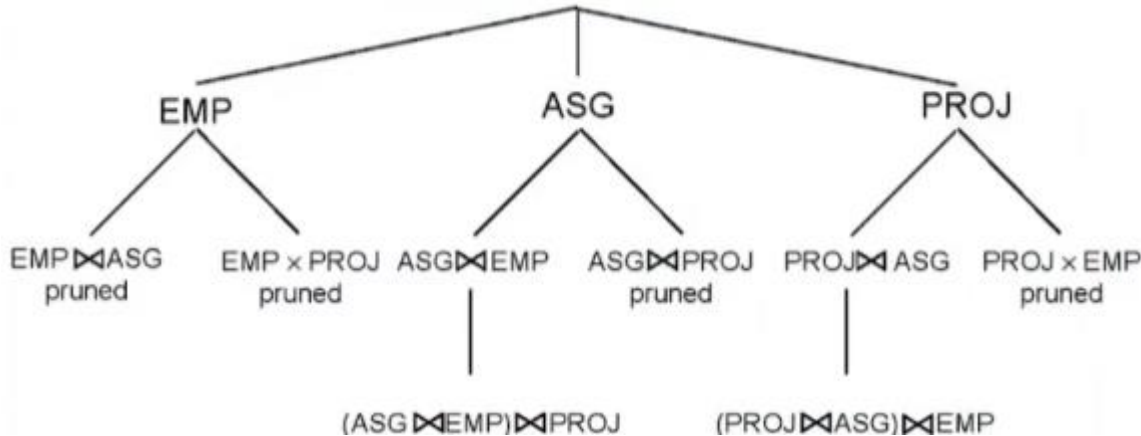
Join Graph:



Step 1 - Select the best single-relation access paths

- EMP: sequential scan (because there is no selection condition on EMP)
- ASG: sequential scan (same)
- PROJ: index on PNAME (because there is a selection on PROJ based on PNAME=CAD/CAM)

Step 2- Select the best join ordering for each relation



- The operations marked pruned are dynamically eliminated.
- (EMP × PROJ) and (PROJ × EMP) are pruned as cartesian products have no common attribute.
- We assume that (EMP ⋈ ASG) and (ASG ⋈ PROJ) have higher cost than (ASG ⋈ EMP) and (PROJ ⋈ ASG) respectively
- So, we get ((ASG ⋈ EMP) ⋈ PROJ) and ((PROJ ⋈ ASG) ⋈ EMP)
- Best total join order is ((PROJ ⋈ ASG) ⋈ EMP), as it uses the indexes best
 - Select PROJ using index on PNAME which is given in question
 - Join with ASG using index on PNO

R* Algorithm

The R* algorithm is a query optimization algorithm which is an extension of the System R query — the distributed version of IBM's System R relational DBMS. It focuses on efficient query processing in distributed environments.

- Only entire relations are stored at each site (no fragmentation or replication).
- Query compilation happens in a distributed way but is controlled by the master site (where query starts).
- The **master site** decides:
 - Where each part of the query runs,
 - How joins are ordered,
 - How data moves between sites.
 - Join order, and
 - Minimizing data transfer between sites.
- **Local sites handle:**
 - **Accessing** data efficiently,
 - Doing joins on data stored locally.

Assumptions of R*:

- Distributed database with relations located at multiple sites.
- Communication cost between sites is significant.
- Goal is to minimize total cost (I/O + communication).

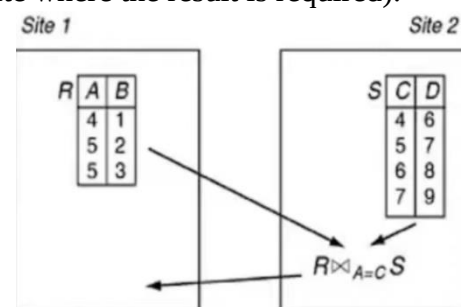
Key Features of R*:

- **Cost-based:** Selects plans that minimize total I/O, CPU, and communication cost.
- **Dynamic programming:** Builds optimal global plans from optimal subplans.
- **Semi-join strategy:** Uses semi-joins to reduce data transfer before full joins.
- **Data localization:** Prefers executing operations at data source sites.
- **Communication-efficient:** Minimizes inter-site data movement.
- **Statistics-driven:** Uses relation size, selectivity, and metadata for planning.
- **Distributed join support:** Handles hash, sort-merge, and nested-loop joins across sites.
- **Site autonomy:** Works with independently managed and autonomous sites.
- **Scalable:** Efficient even with many sites and large datasets.
- **Multi-query optimization:** Reuses intermediate results across queries.
- **Adaptive:** Can adjust execution based on run-time conditions (in advanced versions).

2 methods in R* for inter-site data transfer

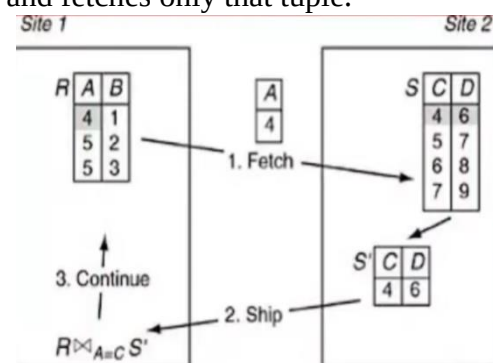
1. Ship Whole: Instead of fetching only required rows, transfer entire relations (tables) from remote sites to the site where the query will be executed (typically the site where the result is required).

- **Pros:**
 - Simple to implement.
 - Reduces the number of messages (just one large transfer).
 - Better if relations are small.
 - Larger data transfer
- **Cons:**
 - Very costly if the table is large.
 - Wastes bandwidth by transferring unnecessary data (i.e., rows not needed for the result).



2. Fetch as Needed (Tuple Shipping): Only the required rows are fetched from the remote sites as needed during query execution. The query is executed in a tuple-at-a-time manner. Whenever the local site needs a tuple from the remote site, it sends a request and fetches only that tuple.

- **Pros:**
 - Avoids unnecessary data transfer.
 - Efficient if only a few tuples are needed from the remote site.
 - Better if relations are large and selectivity is good.
- **Cons:**
 - High communication overhead due to many small messages.
 - Slower due to multiple network round-trips.



Join Processing and Cost Formulas in R* Algorithm

Assumptions:

- R= external relation
- S=internal relation
- **card(R):** Number of tuples in external relation R
- **card(S):** Number of tuples in internal relation S
- **s:** Average number of S tuples matching one R tuple
- **length(A):** Size of the join attribute A
- **length(S):** Size of a tuple in S
- **CT(x):** Communication time of transferring x
- **LT(x):** Local processing (I/O + CPU) for x

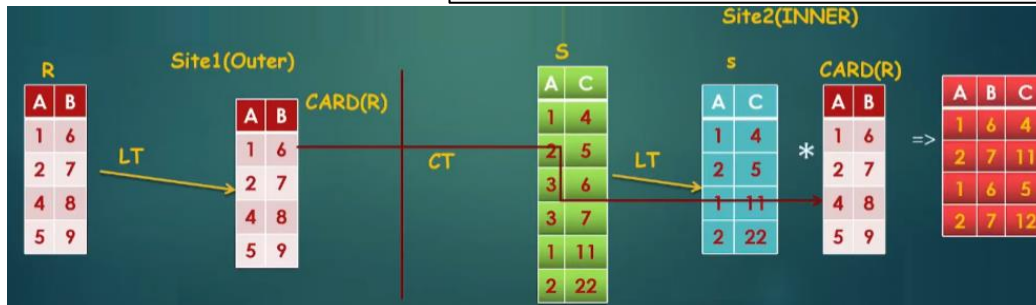
Average number of S tuples matching one R tuple =

$$s = \frac{\text{card}(S \bowtie_A R)}{\text{card}(R)}$$

◆ Strategy 1: Ship R(external) to S's(internal) site

External relation R is sent to the internal relation S's site.

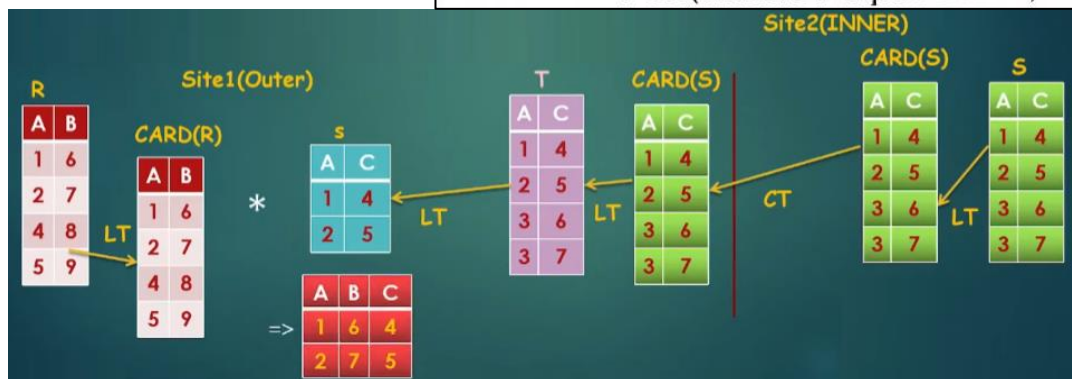
$$\begin{aligned} \text{Total_cost} = & LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\ & + CT(\text{size}(R)) \\ & + LT(\text{retrieve } s \text{ tuples from } S) * \text{card}(R) \end{aligned}$$



◆ Strategy 2: Ship S to R's site

Internal relation S is sent to R's site and stored temporarily.

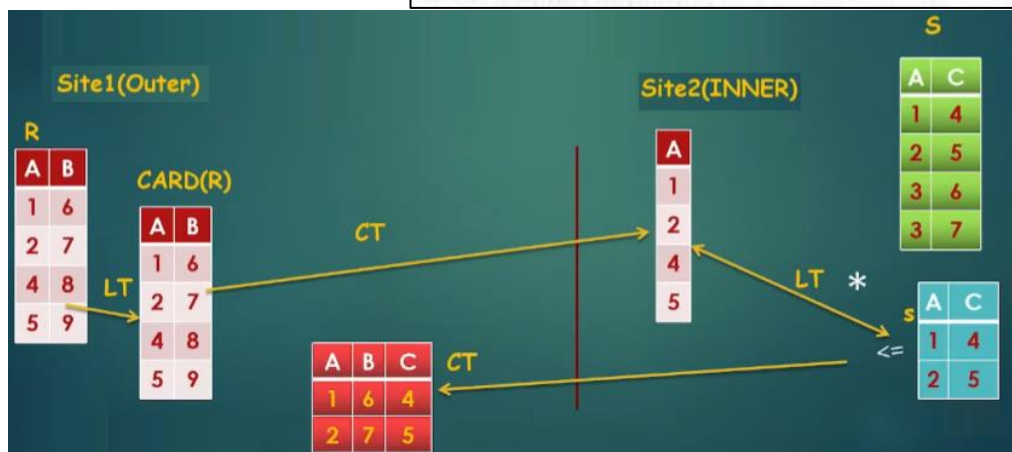
$$\begin{aligned} \text{Total_cost} = & LT(\text{retrieve } \text{card}(S) \text{ tuples from } S) \\ & + CT(\text{size}(S)) \\ & + LT(\text{store } \text{card}(S) \text{ tuples in } T) \\ & + LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\ & + LT(\text{retrieve } s \text{ tuples from } T) * \text{card}(R) \end{aligned}$$



◆ Strategy 3: Fetch S tuples as needed

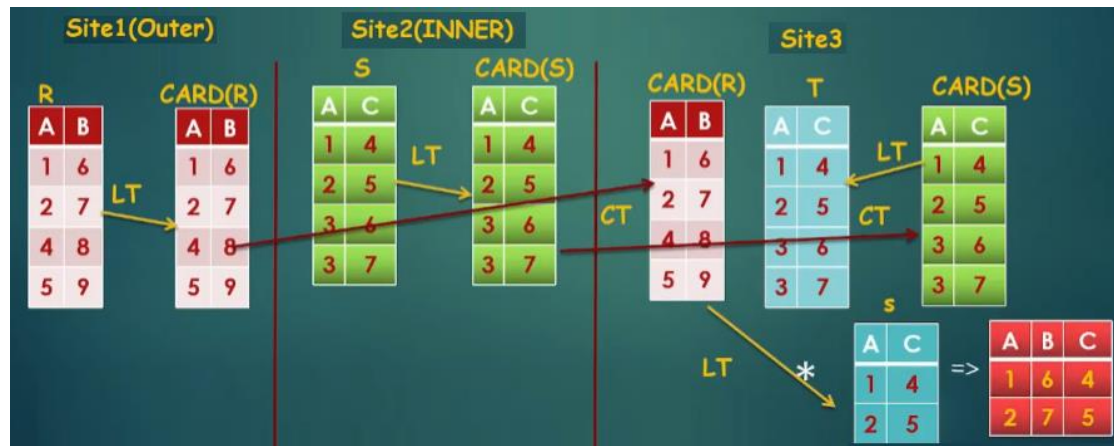
For each R tuple, fetch matching S tuples individually from S's site.

$$\begin{aligned} \text{Total_cost} = & LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\ & + CT(\text{length}(A)) * \text{card}(R) \\ & + LT(\text{retrieve } s \text{ tuples from } S) * \text{card}(R) \\ & + CT(s * \text{length}(S)) * \text{card}(R) \end{aligned}$$



◆ **Strategy 4: Move both to a third site**
R and S are both shipped to a third site and joined there.

$$\begin{aligned}
 \text{Total_cost} = & LT(\text{retrieve } \text{card}(S) \text{ tuples from } S) \\
 & + CT(\text{size}(S)) \\
 & + LT(\text{store } \text{card}(S) \text{ tuples in } T) \\
 & + LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\
 & + CT(\text{size}(R)) \\
 & + LT(\text{retrieve } s \text{ tuples from } T) * \text{card}(R)
 \end{aligned}$$



Strategy	Comm. Cost	Local Processing	Total Cost Trend	Best When	Worst When
Ship R to S	Moderate (CT(size(R)))	Moderate	Usually lower	R is small, S is large, and s is small	R is large
Ship S to R	High (CT(size(S)))	High (store + join)	Often higher than 1	S is very small and cheap to store	S is large
Fetch S as needed	Very high (many small CTs)	Moderate	Highest total cost	s is very small and card(R) is low	s or card(R) is large
Move both to third site	Highest (move both R and S)	Highest	Very high	Rarely best; only if third site is fastest or local load balancing is needed	Usually impractical for large data

Example:

PROJ(External)			ASG(Internal)			
PNO	PNAME	Budget	ENO	PNO	RESP	DUR
Strategy	When Preferred		Remarks			
Strategy 4 (Move both to third site)	Almost never unless third site is extremely fast or needed for load balancing		✗ Highest cost – both relations transferred			
Strategy 2 (Ship internal ASG to external PROJ)	✓ Best when PROJ is much larger than ASG		✓ Minimizes communication if ASG is small and local processing is not high			
Strategy 1 & 3 (Ship PROJ to ASG or fetch ASG as needed)	Used when Strategy 2 isn't optimal		⚖️ Local processing cost is similar for both; choose based on s and comm. overhead			

Aspect	System R	R*
Environment	Centralized DBMS	Distributed DBMS
Optimization Goal	Minimize I/O and CPU cost	Minimize I/O, CPU, and communication
Data Location	Single site	Multiple sites
Join Ordering	Dynamic programming based	Dynamic programming with site awareness
Semijoin Usage	Not used	Used to reduce data transfer
Access Path Choice	Based on local statistics	Based on site and network stats
Cost Estimation	Local costs only	Local + transfer cost considered
Intermediate Results	Local storage	Might be transferred between sites
Communication Cost	Ignored	Explicitly minimized

Hill Climbing

- **Type:** Heuristic (greedy) search method
- **Approach:** Starts with an initial query plan, then **iteratively applies transformations** to move to a **neighbor plan** with **lower estimated cost**.
- **Goal:** Reach a **locally optimal plan** (minimum cost)
- **Limitation:** May get stuck in **local minima** (not guaranteed to find global optimum)
- **Usage:** Fast and simple, used when exhaustive search (like dynamic programming) is too costly.

SDD-1 Algorithm

- **Type:** Rule-based heuristic optimization algorithm
- **Used in:** SDD-1 system — a pioneering distributed database system
- **Approach:**
 - Converts high-level SQL query to an **algebraic representation**
 - Applies **semantic query transformations**
 - Optimizes **data localization, join ordering, and data transfer**
- **Key Features:**
 - Emphasizes **data localization** (execute as much as possible at the data site)
 - Supports **semi-joins** to reduce data transfer
- **Goal:** Minimize **total cost** = local processing + communication cost

Distributed Cost Model

- The cost of a distributed execution strategy can be expressed with respect to either the total time or the response time.
- Total time is sum of all time components.

$$\text{Total_Time} = T_{\text{CPU}} \times \# \text{insts} + T_{\text{I/O}} \times \# \text{I/Os} + T_{\text{MSG}} \times \# \text{msgs} + T_{\text{TR}} \times \# \text{bytes}$$

- Response time is the elapsed time from the initiation to the completion of the query

$$\text{Response_Time} = \max (T_{\text{CPU}} \times \# \text{insts}, T_{\text{I/O}} \times \# \text{I/Os}, T_{\text{MSG}} \times \# \text{msgs}, T_{\text{TR}} \times \# \text{bytes})$$

- RT based on communication time

$$\text{Communication_Time} = T_{\text{MSG}} \times \# \text{msgs} + T_{\text{TR}} \times \# \text{bytes}$$

$$\text{Response_Time} = \max(T_{\text{MSG}} + T_{\text{TR}} \times x, T_{\text{MSG}} + T_{\text{TR}} \times y)$$

T_{CPU} = time of a CPU instruction

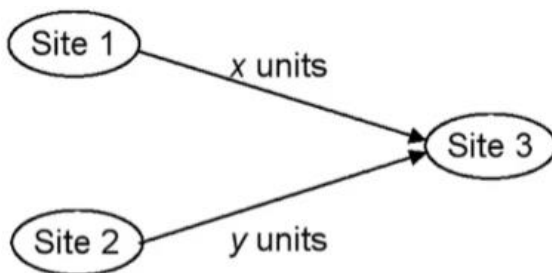
$T_{\text{I/O}}$ = time of a disk I/O

The communication time is depicted by the two last components.

T_{MSG} = fixed time of initiating and receiving a message

T_{TR} = time it takes to transmit a data unit from one site to another

Example:



find total time and response time

Answer:

We see no CPU time or IO time is mentioned here, so,

$$\begin{aligned} \text{Total time} &= \# \text{msg} * T_{\text{msg}} + T_{\text{tr}} * \# \text{bytes} \\ &= 2T_{\text{msg}} + T_{\text{tr}}(x+y) \end{aligned}$$

$$\text{Communication time for Site1 to Site3} = 1 * T_{\text{msg}} + T_{\text{tr}} * x$$

$$\text{Communication time for Site2 to Site3} = 1 * T_{\text{msg}} + T_{\text{tr}} * y$$

$$\text{Response time} = \max(T_{\text{msg}} + T_{\text{tr}} * x, T_{\text{msg}} + T_{\text{tr}} * y)$$

Factors Governing Query Optimization

1. **Data distribution** – Location and fragmentation of data across sites
2. **Network latency** – Delay in data transmission between sites
3. **Communication cost** – Message startup and data transfer time
4. **Local processing cost** – CPU and disk I/O time at individual sites
5. **Query structure** – Complexity, joins, subqueries, and nesting
6. **Site autonomy** – Degree of control each site has over its data
7. **Data replication** – Multiple copies of data affecting choice of source
8. **System load** – Current workload and resource availability at sites
9. **Access methods** – Availability of indexes or access paths
10. **Intermediate result size** – Volume of data generated during processing
11. **Join ordering** – Sequence in which relations are joined
12. **Parallelism** – Potential to execute parts of the query concurrently

Centralized and Decentralized Query Optimization

1. Centralized Query Optimization

- a. Done at a single central site
- b. Has global knowledge of data distribution and system state
- c. Generates one global execution plan
- d. Simpler control, but may be a bottleneck
- e. Suitable for tightly coupled systems
- f. Algorithms: INGRES algorithm, System R algorithm

2. Decentralized Query Optimization

- a. Done locally at each site
- b. Each site optimizes based on partial/local knowledge
- c. Results are combined for overall execution
- d. Scalable and avoids central bottlenecks
- e. Suitable for loosely coupled or federated systems
- f. Algorithm: R* algorithm, SDD-1 algorithm, distributed INGRES algorithm